

Pontificia Universidad Católica de Chile
Escuela de Ingeniería
Departamento de Ciencia de la Computación



IIC2613 - Inteligencia Artificial

Kernel SVM

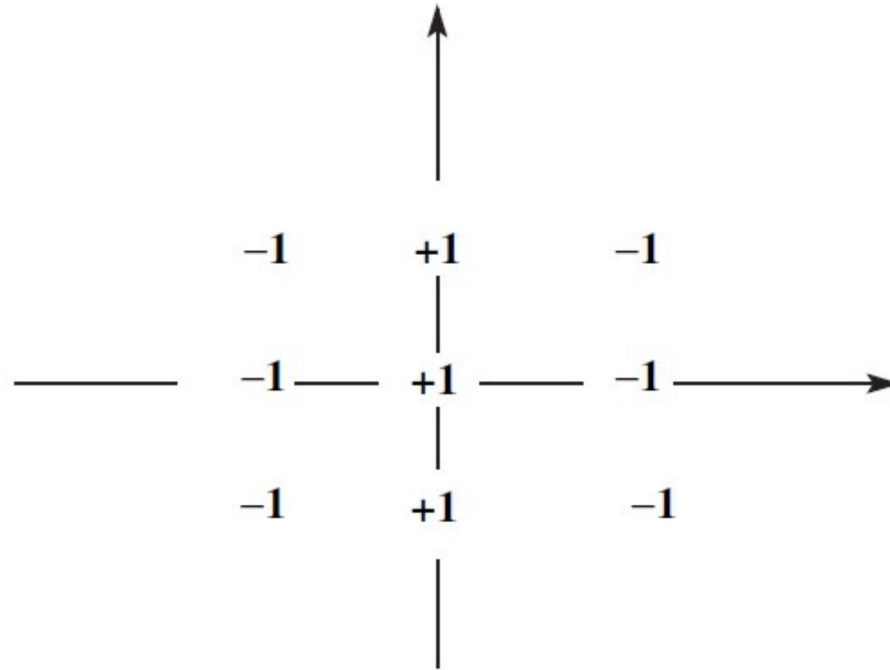
Denis Parra
Dpto. Ciencia de la Computación

Empecemos esta clase con un poco de código (Carpeta archivos de Canvas)

- Hard-margin SVM y vectores de soporte
- Soft-margin SVM
- Problema de clasificación no lineal

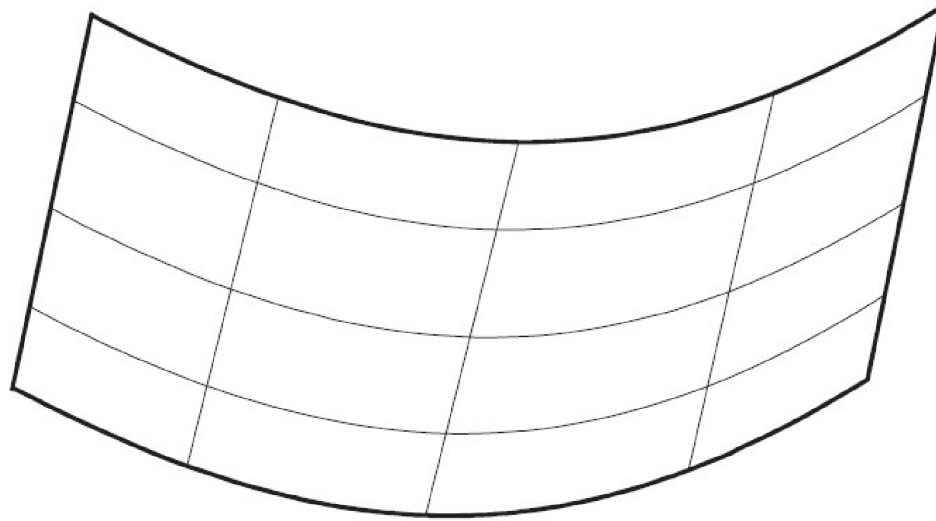


SVM es poderoso, pero sigue siendo un clasificador lineal



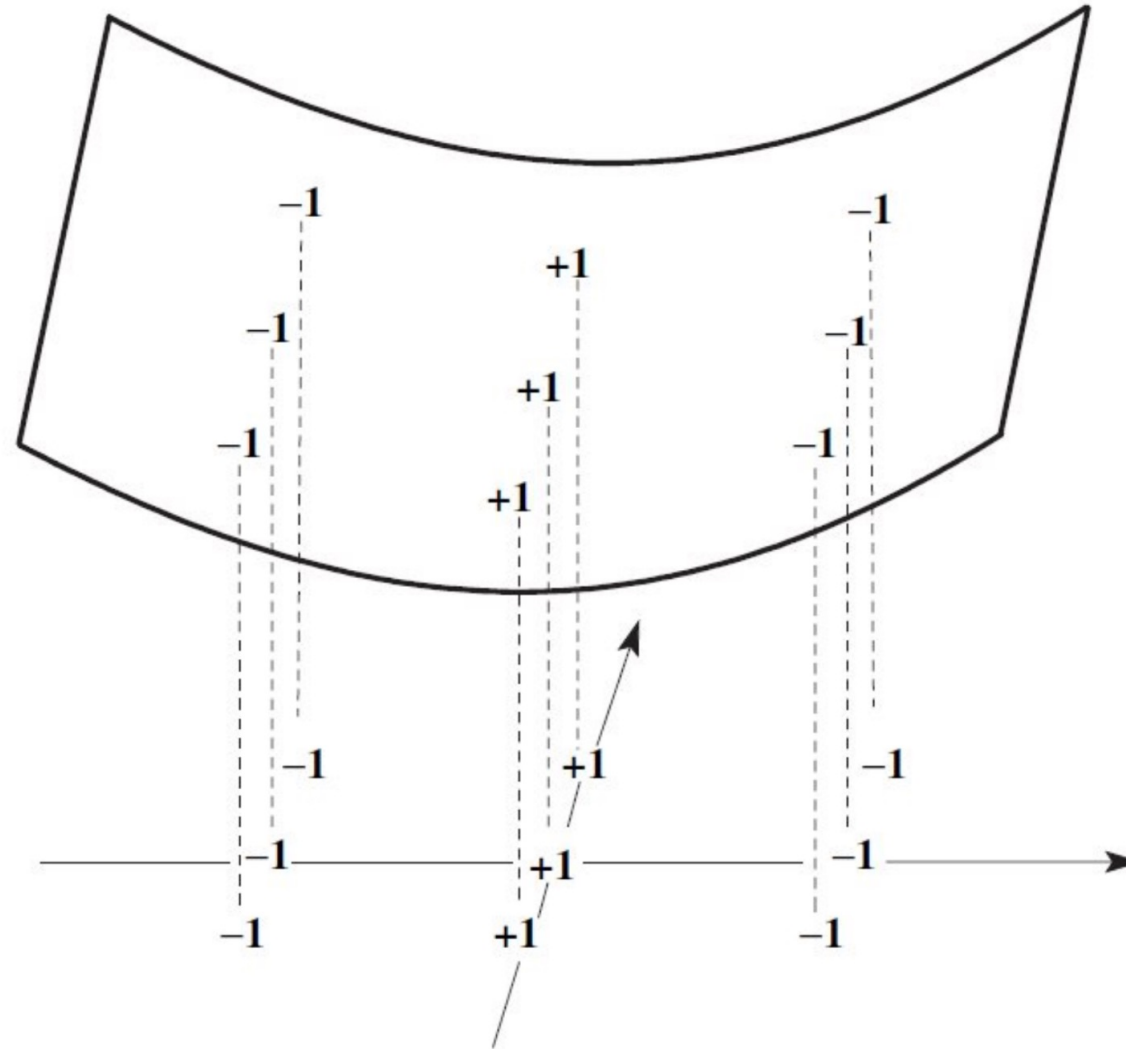
¿Es posible solucionar este problema con un SVM?

Una solución es generar un cambio de variables
(transformación del espacio de características)

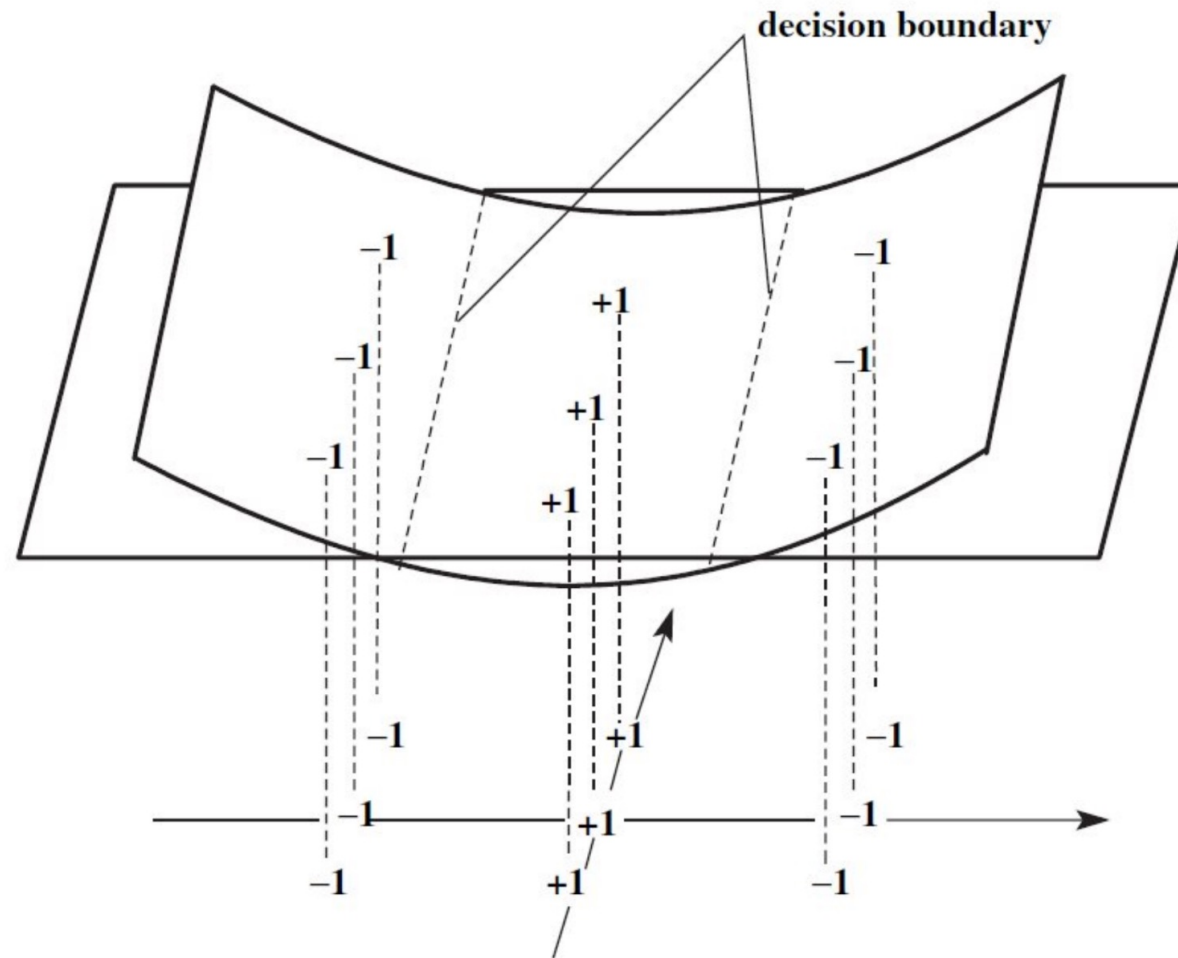


Espacio de características $f(x_1, x_2) = x_1^2$

Una solución es generar un cambio de variables
(transformación de espacio de características)



No es muy distinto a regresión lineal con polinomios de mayor grado

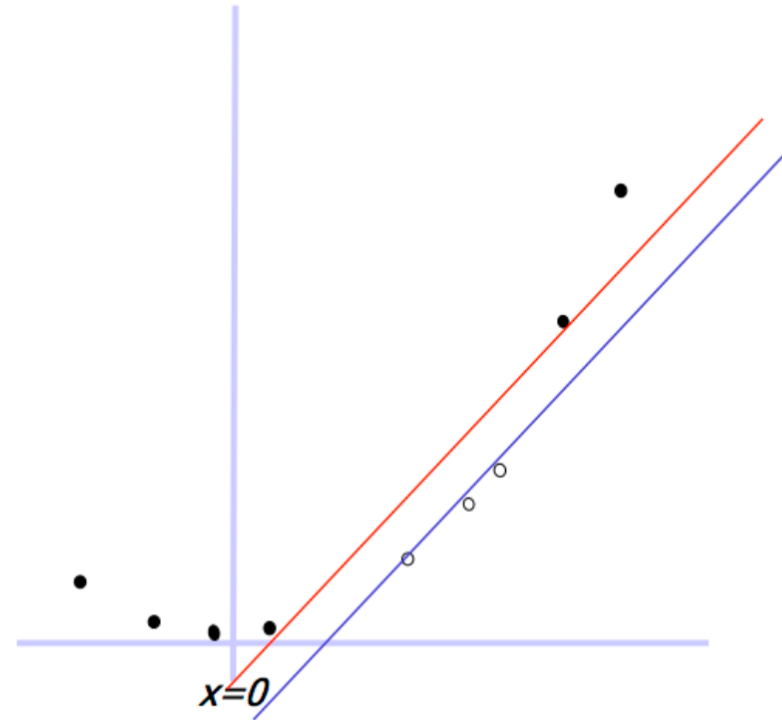


Otro ejemplo

$\mathbf{x}$

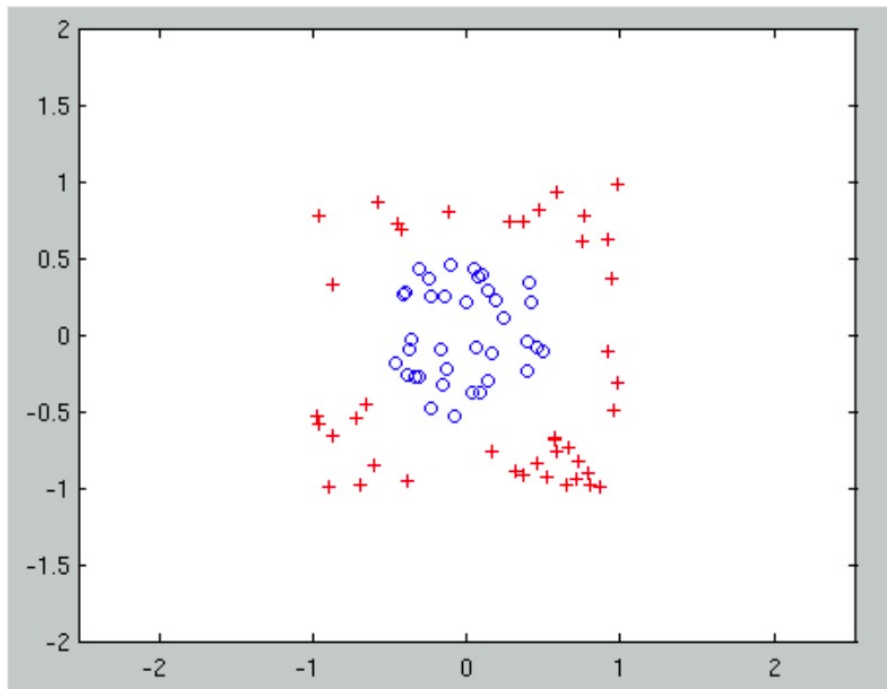
$\rightarrow$

$$\phi(\mathbf{x}) = (\mathbf{x}, \mathbf{x}^2)$$

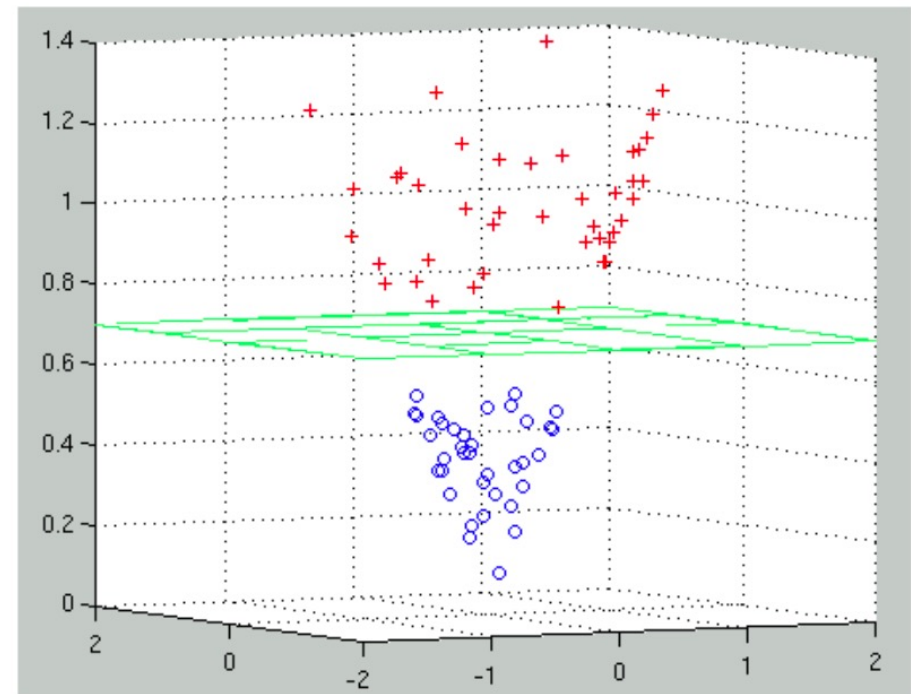


Otro ejemplo

$$(\mathbf{x}_1, \mathbf{x}_2)$$



$$\rightarrow \phi(\mathbf{x}) = (\mathbf{x}_1, \mathbf{x}_2, \sqrt{\mathbf{x}_1^2 + \mathbf{x}_2^2})$$



La pregunta es entonces, como podemos hacer más sistemático este mecanismo

- El ir probando grado a grado una expansión polinomial, claramente no escala en tiempo (ni de uno, ni del computador).
- Además, espacios de *features* de dimensionalidad muy grande son problemáticos de manejar (espacio + tiempo de ejecución).
- Peor aún, la distancia euclidiana entre vectores deja de tener sentido (todos los puntos están en promedio igual de cerca -> maldición de la dimensionalidad).
- ¿Estamos forzados a vivir en un espacio de superficies de decisión lineal?

¿Qué es un Kernel?

- Supongamos que tenemos una función $h(x)$, que lleva un vector x a un nuevo espacio de dimensionalidad arbitraria (potencialmente infinita).
- Imaginemos que, para cada par de vectores, $h(x_i)$ y $h(x_j)$, calculamos su producto punto y lo guardamos en la posición i, j de una matriz M .
- **Acá viene la magia:** si M es **positiva definida** ($z^T M z > 0, \forall z \neq \vec{0}$), entonces siempre existe una función $K(x_i, x_j)$, llamada *kernel*, que toma dos vectores, x_i y x_j , y retorna el valor del producto punto entre $h(x_i)$ y $h(x_j)$, sin necesidad de conocer la función $h(x)$.

¿Qué es un Kernel?

- Supongamos que tenemos una función $h(x)$, que lleva un vector x a un nuevo espacio de dimensionalidad arbitraria (potencialmente infinita).
- Imaginemos que, para cada par de vectores, $h(x_i)$ y $h(x_j)$, calculamos su producto punto y lo guardamos en la posición i, j de una matriz M .

- **Acá viene la magia:** si M es **positiva definida** ($z^T M z > 0, \forall z \neq \vec{0}$), entonces siempre existe una función $K(x_i, x_j)$, llamada *kernel*, que toma dos vectores, x_i y x_j , y retorna el valor del producto punto entre $h(x_i)$ y $h(x_j)$, sin necesidad de conocer la función $h(x)$.



- ***¿Qué significa esto?***
- ***¡Veamos un ejemplo!***

Ejemplo de Claude

- Uno podría suponer "Primero defino $\mathbf{h}(\mathbf{x})$, luego calculo $K(\mathbf{x}_i, \mathbf{x}_j) = \langle \mathbf{h}(\mathbf{x}_i), \mathbf{h}(\mathbf{x}_j) \rangle$ "
- **En cambio**, primero defino K directamente en el espacio de entrada, luego demuestro que existe algún $\mathbf{h}()$ tal que $K(\mathbf{x}_i, \mathbf{x}_j) = \langle \mathbf{h}(\mathbf{x}_i), \mathbf{h}(\mathbf{x}_j) \rangle$

Ejemplo concreto: Kernel Polinomial

Sea $\mathbf{x} = (x_1, x_2) \in \mathbb{R}^2$. Defino directamente:

$$K(\mathbf{x}, \mathbf{z}) = (\mathbf{x}^\top \mathbf{z})^2 = (x_1 z_1 + x_2 z_2)^2$$

Expando algebraicamente:

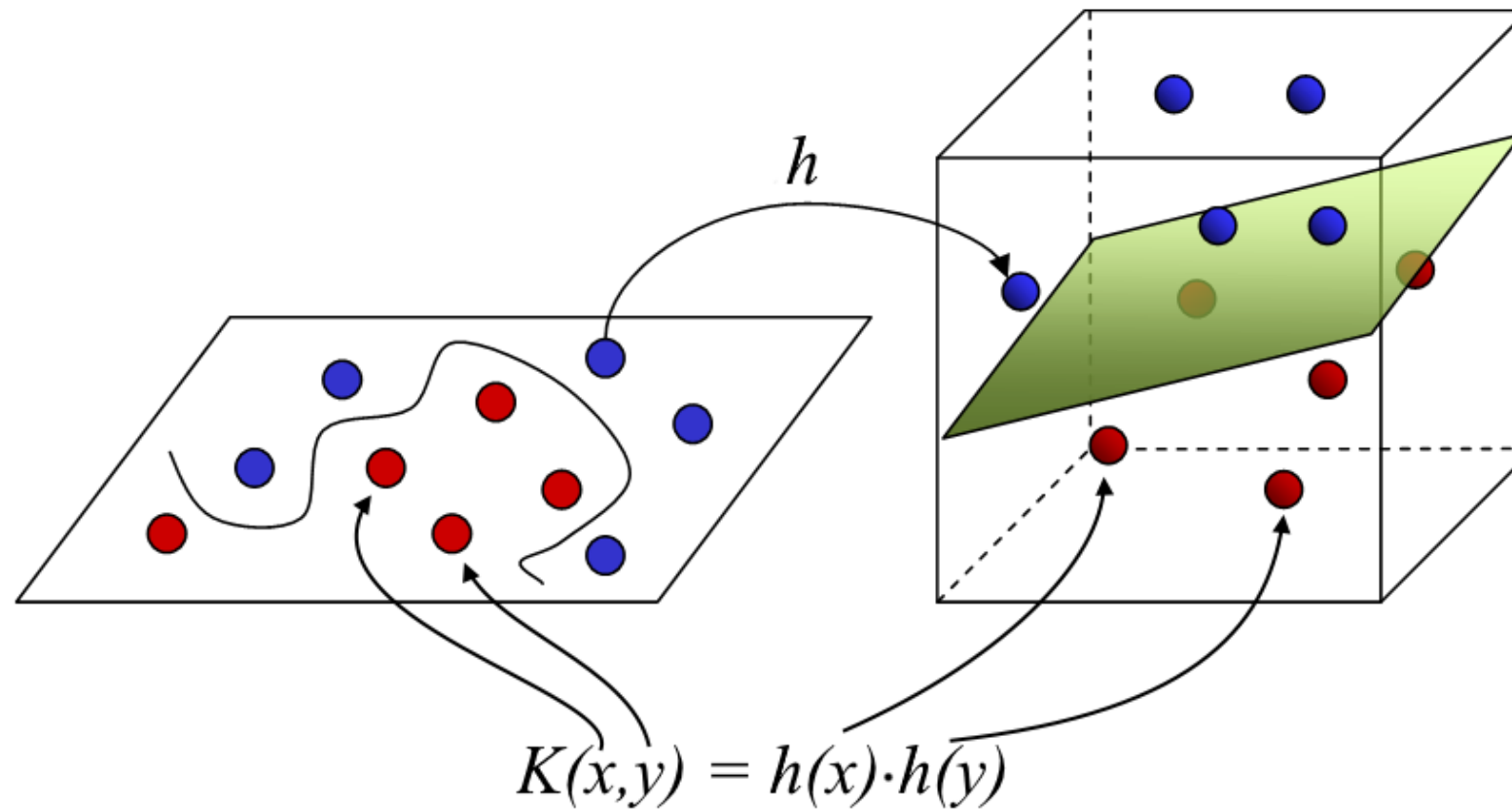
$$= x_1^2 z_1^2 + x_2^2 z_2^2 + 2 x_1 x_2 z_1 z_2$$

Eso se puede reescribir como un producto punto:

$$= \underbrace{\begin{pmatrix} x_1^2 \\ x_2^2 \\ \sqrt{2} x_1 x_2 \end{pmatrix}}_{\mathbf{h}(\mathbf{x})} \cdot \underbrace{\begin{pmatrix} z_1^2 \\ z_2^2 \\ \sqrt{2} z_1 z_2 \end{pmatrix}}_{\mathbf{h}(\mathbf{z})}$$

Así que $\mathbf{h}(\mathbf{x}) = (x_1^2, x_2^2, \sqrt{2} x_1 x_2)$ es el mapa de características que corresponde a K — pero lo *descubrimos* expandiendo K , no al revés. Nunca necesitamos saber $\mathbf{h}$ de antemano para usar K .

Visualmente, la **ventaja** de los **kernels** toma más sentido



¿Y qué tiene que ver esto con los SVM?

Recordemos brevemente la resolución del problema de optimización asociado a un SVM

$$\min_{\beta, \beta_0} \frac{1}{2} \|\beta\|^2$$

$$\text{subject to } y_i(x_i^T \beta + \beta_0) \geq 1, \quad i = 1, \dots, N$$



Lagrangiano

$$L_P = \frac{1}{2} \|\beta\|^2 - \sum_{i=1}^N \alpha_i [y_i(x_i^T \beta + \beta_0) - 1]$$



Imponiendo las condiciones de KKT, más algo de álgebra

$$\max_{\alpha} \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i=1}^N \sum_{k=1}^N \alpha_i \alpha_k y_i y_k x_i^T x_k$$

$$\text{subject to } \alpha_i \geq 0 \text{ and } \sum_{i=1}^N \alpha_i y_i = 0 \text{ and } \alpha_i [y_i(x_i^T \beta + \beta_0) - 1] = 0 \quad \forall i$$

Podemos incorporar esto en los SVM,
mediante algo conocido como el *kernel trick*

$$L_D = \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i=1}^N \sum_{i'=1}^N \alpha_i \alpha_{i'} y_i y_{i'} \hat{x}_i^T \hat{x}_{i'}$$



$$L_D = \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i=1}^N \sum_{i'=1}^N \alpha_i \alpha_{i'} y_i y_{i'} \langle \hat{x}_i, \hat{x}_{i'} \rangle$$



$$L_D = \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i=1}^N \sum_{i'=1}^N \alpha_i \alpha_{i'} y_i y_{i'} \langle h(x_i), h(x_{i'}) \rangle$$



$$L_D = \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i=1}^N \sum_{i'=1}^N \alpha_i \alpha_{i'} y_i y_{i'} K(x_i, x_{i'})$$

Podemos incorporar esto en los SVM,
mediante algo conocido como el *kernel trick*

$$f(x) = h(x)^T \beta + \beta_0$$

$$\beta = \sum_{i=1}^N \alpha_i y_i h(x_i)$$

Podemos incorporar esto en los SVM,
mediante algo conocido como el *kernel trick*

$$\begin{aligned} f(x) &= h(x)^T \beta + \beta_0 \\ &= \sum_{i=1}^N \alpha_i y_i \langle h(x), h(x_i) \rangle + \beta_0 \end{aligned} \qquad \beta = \sum_{i=1}^N \alpha_i y_i h(x_i)$$

Podemos incorporar esto en los SVM,
mediante algo conocido como el *kernel trick*

$$\begin{aligned} f(x) &= h(x)^T \beta + \beta_0 \\ &= \sum_{i=1}^N \alpha_i y_i \langle h(x), h(x_i) \rangle + \beta_0 \\ &= \sum_{i=1}^N \alpha_i y_i K(x, x_i) + \beta_0 \end{aligned} \qquad \beta = \sum_{i=1}^N \alpha_i y_i h(x_i)$$

Podemos incorporar esto en los SVM,
mediante algo conocido como el *kernel trick*

$$\begin{aligned} f(x) &= h(x)^T \beta + \beta_0 \\ &= \sum_{i=1}^N \alpha_i y_i \langle h(x), h(x_i) \rangle + \beta_0 \\ &= \sum_{i=1}^N \alpha_i y_i K(x, x_i) + \beta_0 \end{aligned} \qquad \beta = \sum_{i=1}^N \alpha_i y_i h(x_i)$$

- Como siempre operamos sobre el producto punto entre vectores $h(x)$, y no sobre los $h(x)$ de manera individual, también podemos sustituirlos por la aplicación del mismo *kernel* $K(x, y)$.
- Esto nos permite facilitar la búsqueda de la solución, ya que mientras más dimensiones hay, más posibilidades existen de encontrar un hiperplano óptimo

Intuitivamente, ¿qué mide un *kernel*?

Intuitivamente, un *kernel* mide la similitud entre dos vectores (pero en un espacio distinto)

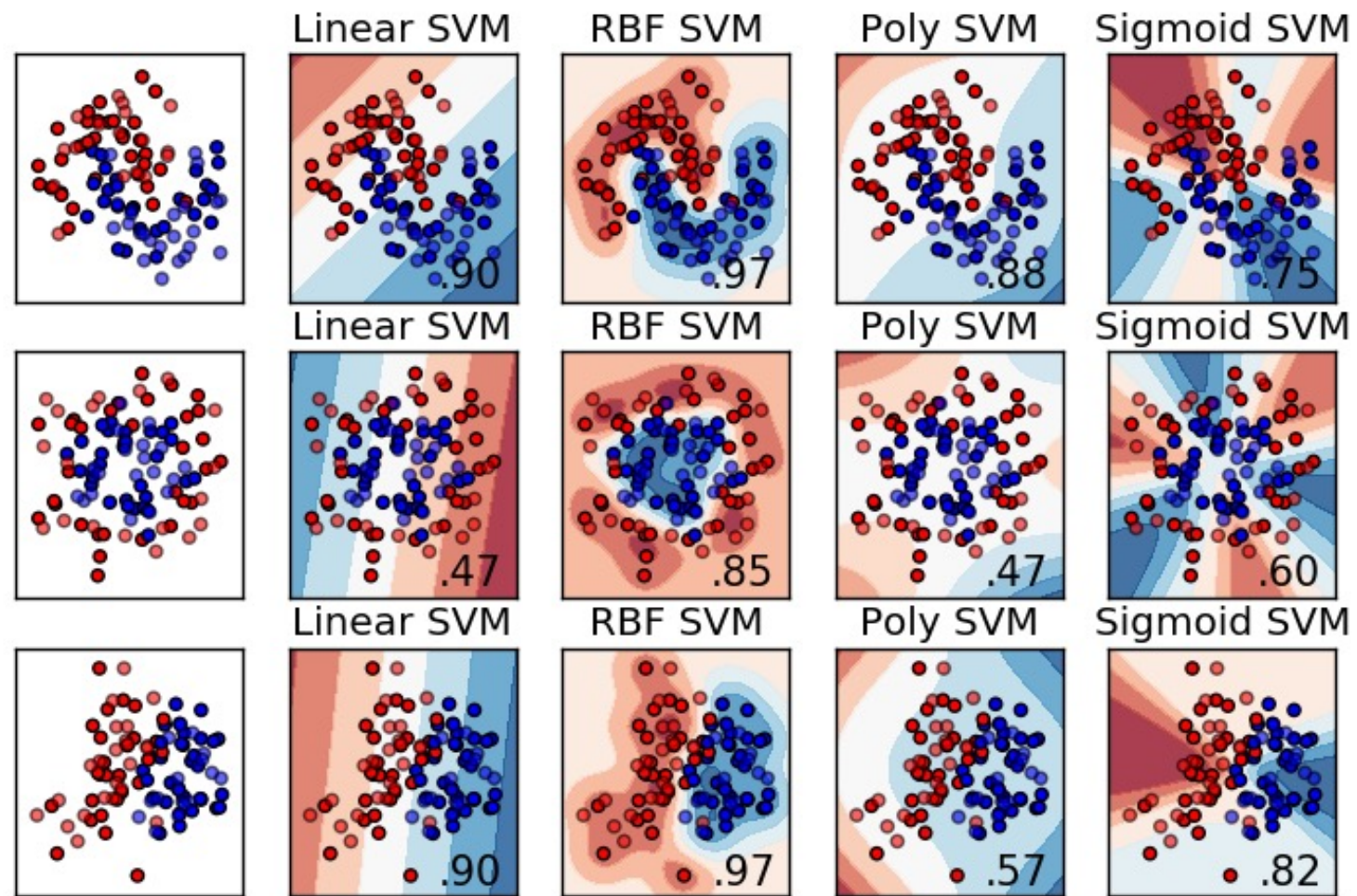
$$f(x) = \sum_{i=1}^N \alpha_i y_i K(x, x_i) + \beta_0$$

*d*th-Degree polynomial: $K(x, x') = (1 + \langle x, x' \rangle)^d$,

Radial basis: $K(x, x') = \exp(-\gamma \|x - x'\|^2)$,

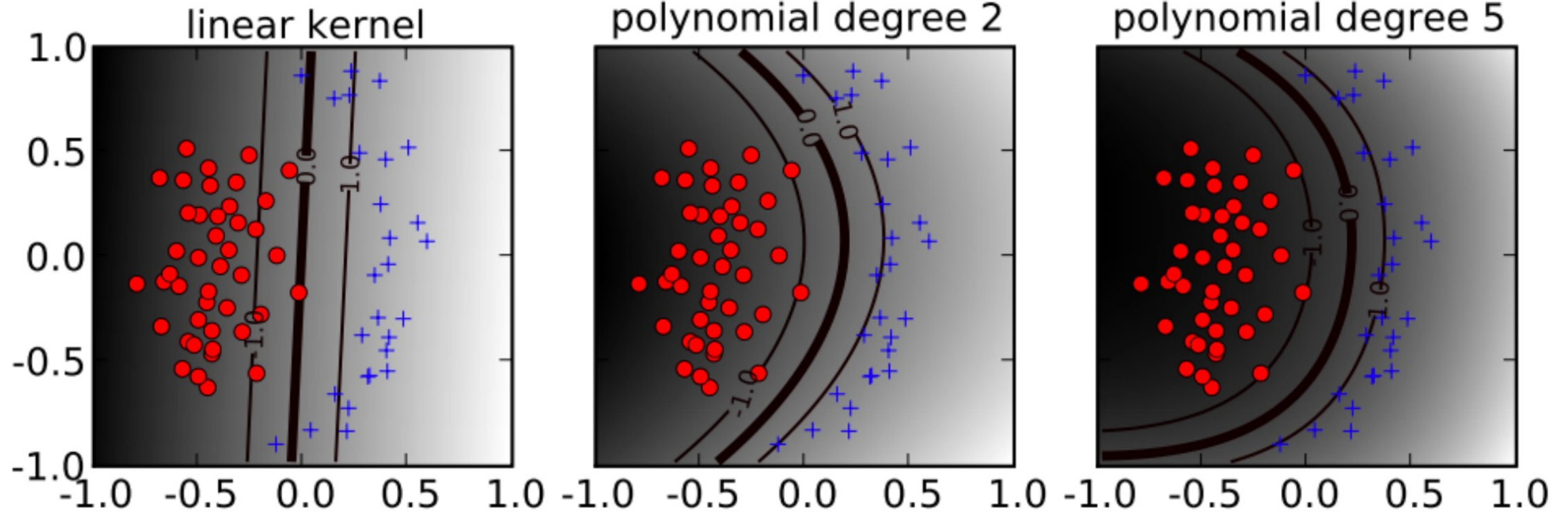
Neural network: $K(x, x') = \tanh(\kappa_1 \langle x, x' \rangle + \kappa_2)$.

¿Cuál es el *kernel* de los SVM que vimos inicialmente?



Uso de Kernels: Kernel Polinomial

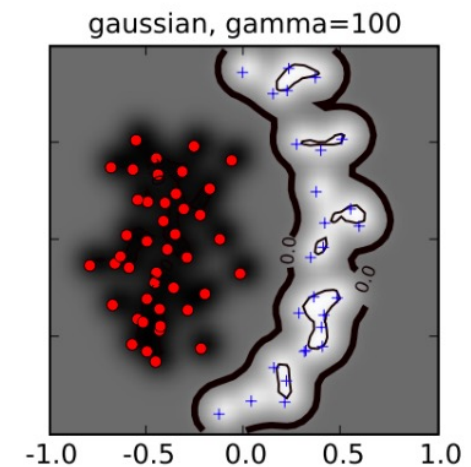
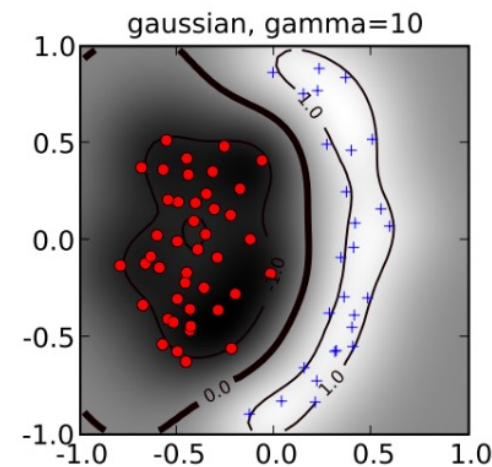
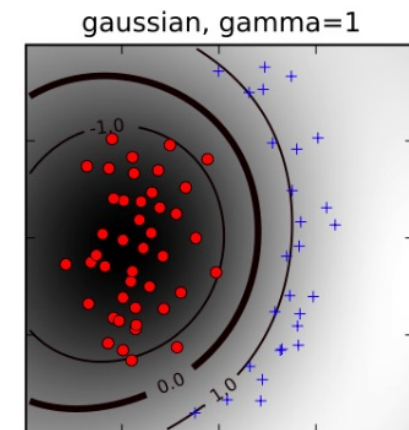
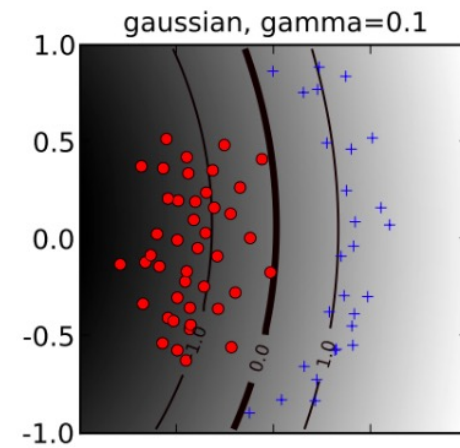
$$K(x, x') = (1 + \langle x, x' \rangle)^d$$



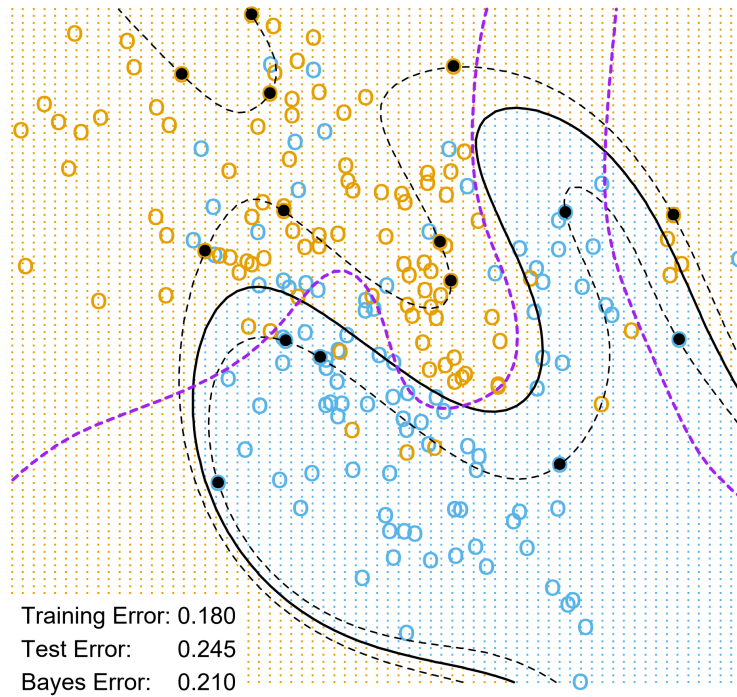
Uso de Kernerls: Kernel RBF

$$K(x, x') = \exp(-\gamma \|x - x'\|^2)$$

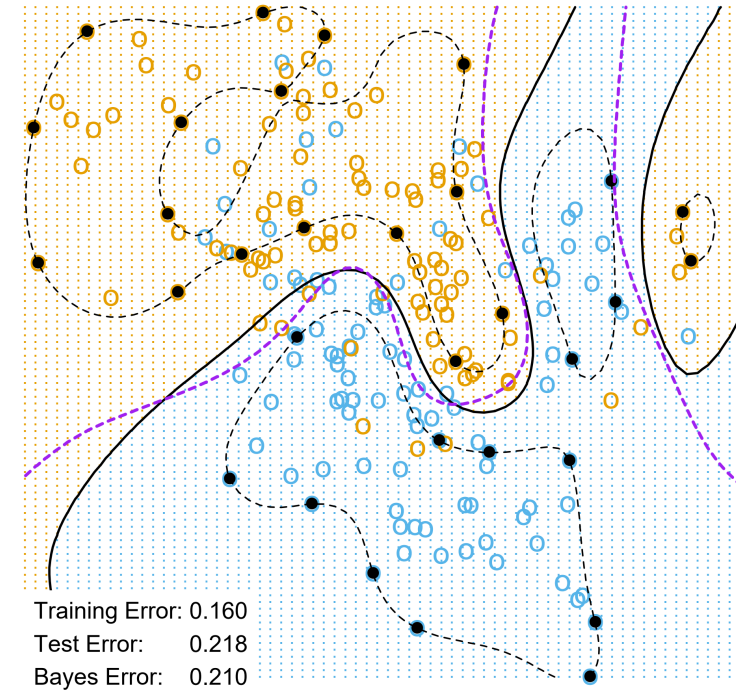
- Para valores pequeños de γ (arriba a la izquierda) la frontera de decisión es casi lineal.
- A medida que γ aumenta, la flexibilidad de la frontera de decisión aumenta.
- Valores grandes de γ llevan a sobreajuste (abajo).



SVM - Degree-4 Polynomial in Feature Space



SVM - Radial Kernel in Feature Space

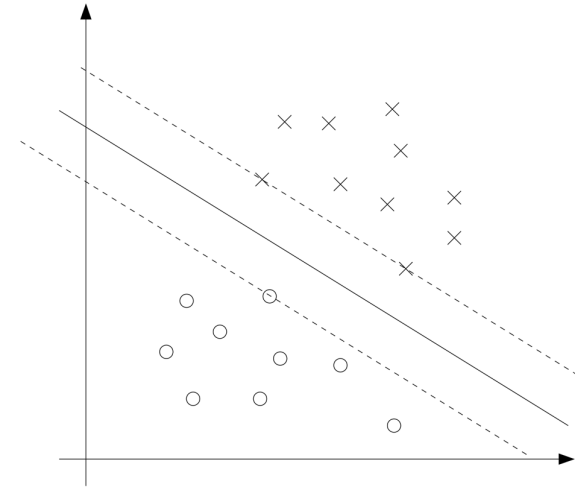


Un par de preguntas para terminar Kernel SVMs

- ¿Cuáles son los vectores de soporte en este caso?
- ¿Cómo se ve el margen en este caso?

SVMs continúan siendo relevantes en *machine learning*

- SVMs son de los algoritmos *off-the-shelf* con mejor rendimiento.
- Simpleza, concepto de margen e incorporación de *kernels* son sus grandes fortalezas.
- Han perdido importancia durante la última década con respecto a las técnicas modernas de Deep Learning y de ensambles.



Ejemplos de código disponibles en Canvas

- Hard-margin SVM y vectores de soporte
- Soft-margin SVM
- **Kernel SVM**
- Visualización de kernels



Pontificia Universidad Católica de Chile
Escuela de Ingeniería
Departamento de Ciencia de la Computación



Fundamentos de Machine Learning

Kernel SVM

Denis Parra
Dpto. Ciencia de la Computación